

```
!git clone https://github.com/stiflet/CandidateProject.git
!hf download Stiflet/WeatherData --repo-type=dataset --local-dir=Weather_Models
```

```
Cloning into 'CandidateProject'...
remote: Enumerating objects: 43, done.
remote: Counting objects: 100% (43/43), done.
remote: Compressing objects: 100% (24/24), done.
remote: Total 43 (delta 12), reused 38 (delta 7), pack-reused 0 (from 0)
Receiving objects: 100% (43/43), 4.69 MiB | 8.73 MiB/s, done.
Resolving deltas: 100% (12/12), done.
A new version of huggingface_hub (1.9.2) is available! You are using version 1.8.0.
To update, run: pip install -U huggingface_hub

Downloading (incomplete total...): 0.00B [00:00, ?B/s]
Fetching 8 files: 0% 0/8 [00:00<?, ?it/s]Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF
Downloading (incomplete total...): 0% 2.50k/185M [00:00<17:24:50, 2.95kB/s]
Downloading (incomplete total...): 185MB [00:14, 16.9MB/s]
Fetching 8 files: 100% 8/8 [00:14<00:00, 1.78s/it]
Download complete: : 185MB [00:14, 16.9MB/s] /content/Weather_Models
Download complete: : 185MB [00:14, 12.9MB/s]
```

✓ Ensemble Model Output Statistics (EMOS)

EMOS is one of the most commonly used baseline models in the probabilistic forecasting literature.

Overview

EMOS is a method used for estimating a full probability distribution over a continuous variable. It does so by using ensemble forecasts, where the predictive mean is modeled as a function of the ensemble mean and the predictive variance is modeled as a function of the spread between the ensemble members.

Mathematical definition

Let

y = observed value of the target variable.

$x_1, x_2, \dots, x_M$ = ensemble member forecasts of the target variable.

Then for the variable temperature as in the example below, the predictive distribution is often assumed to be:

$y \mid x_1, x_2, \dots, x_M \sim \mathcal{N}(\mu, \sigma^2)$ e.g. Guassian

where the predictive mean μ and predictive variance σ^2 are modeled as

$$\mu = a + b\bar{x}$$

$$\sigma^2 = c + dS^2$$

where

$\bar{x} = \frac{1}{M} \sum_{m=1}^M x_m$ is the ensemble mean.

$S^2 = \frac{1}{M-1} \sum_{m=1}^M (x_m - \bar{x})^2$ is the ensemble variance.

The parameters a,b,c and d are estimated from historical forecast and observation data. This allows EMOS to correct where the forecast is centered and how uncertain it is, which often makes the forecast more reliable than the raw ensemble output.

```
from scipy.optimize import minimize
import matplotlib.pyplot as plt
from scipy.stats import norm
import pandas as pd
import numpy as np

#I have gathered historical forecast and observations of different variables
#for the latitude longitude pair closest to LaGuardia airport in New York City.
#The model used is Global Ensemble Forecast Systems (GEFS).
#The provider had the following available in their zarr library (https://dynamical.org/catalog/noaa-gefs-forecast-35-day/)
#Spatial domain: Global
#Spatial resolution:
#0-240 hours: 0.25 degrees (~20km),
#243-840 hours: 0.5 degrees (~40km)
#Time domain:
#Forecasts initialized 2020-10-01 00:00:00 UTC to Present
#Time resolution:
#Forecasts initialized every 24 hours
#Forecast domain:
#Forecast lead time 0-840 hours (0-35 days) ahead
#Forecast resolution:
```

```
#Forecast step 0–240 hours: 3 hourly,  
#243–840 hours: 6 hourly  
#The data has been cleaned and split up in 4 different parquet files each with 4 different leadtimes.  
#3h  
#6h  
#9h  
#12h  
#The variable used for this EMOS is maximum temperature at 2 meters height, and contains 31 ensemble members.  
df = pd.read_parquet(r"/content/Weather_Models/Preprocessing/EMOS/emos_3h.parquet").dropna().sort_index()  
  
ensemble_members = df.drop('obs', axis=1)  
observations = df['obs']  
df
```

ensemble_member	0	1	2	3	4	5	6	7	8	9	...	22	23	
valid_time														
2020-10-01 03:00:00+00:00	20.00000	19.7500	20.0000	20.0000	20.00000	19.87500	20.12500	20.00000	20.1250	19.87500	...	20.2500	19.8750	20.1
2020-10-02 03:00:00+00:00	20.87500	20.7500	21.2500	21.2500	21.12500	21.00000	21.00000	20.50000	20.6250	20.62500	...	20.8750	20.8750	20.6
2020-10-03 03:00:00+00:00	18.12500	18.2500	18.1250	18.1250	18.37500	18.25000	18.25000	18.25000	18.2500	18.12500	...	18.1250	18.1250	18.3
2020-10-04 03:00:00+00:00	17.62500	17.8750	17.2500	17.1250	17.75000	17.25000	17.62500	18.00000	17.7500	18.00000	...	17.8750	18.1250	17.6
2020-10-05 03:00:00+00:00	17.37500	17.3750	17.2500	17.3750	17.25000	17.25000	17.25000	17.50000	17.2500	17.50000	...	17.5000	17.2500	17.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2026-04-01 03:00:00+00:00	19.87500	20.0000	20.0000	20.2500	20.75000	18.87500	20.25000	19.25000	20.2500	20.50000	...	19.2500	19.2500	20.1
2026-04-02 03:00:00+00:00	15.87500	15.5000	16.3750	15.1875	16.25000	15.75000	16.62500	16.25000	16.7500	16.12500	...	15.8125	16.1250	15.6
2026-04-03 03:00:00+00:00	6.90625	7.1250	7.0000	7.4375	7.40625	7.28125	6.46875	6.78125	7.0000	7.21875	...	7.4375	7.0000	6.1
2026-04-04 03:00:00+00:00	15.75000	15.8125	15.4375	15.8750	15.62500	15.81250	15.56250	15.62500	15.9375	15.68750	...	15.6875	15.9375	15.8
2026-04-05 03:00:00+00:00	8.93750	8.5625	9.0625	9.1875	9.50000	8.75000	8.68750	9.12500	9.3750	8.75000	...	8.5625	9.4375	9.2

2013 rows x 32 columns

```
from scipy.stats import norm
import numpy as np
```

```
def crps_ensemble(obs, ensemble): #CRPS for an ensemble forecast.
    obs = np.asarray(obs) #Convert observations to numpy array.
    ensemble = np.asarray(ensemble) #Convert ensemble forecasts to numpy array.
```

```
term1 = np.mean(np.abs(ensemble - obs[:, None]), axis=1) #Average absolute difference between each observation and its
term2 = 0.5 * np.mean(
    np.abs(ensemble[:, :, None] - ensemble[:, None, :]), #Average pairwise absolute difference between ensemble member
    axis=(1, 2)
)
return term1 - term2 #CRPS formula for an ensemble forecast.
```

```
def crps_gaussian(obs, mu, sigma): #CRPS for a Gaussian predictive distribution.
    z = (obs - mu) / sigma #Standardize the observations relative to the Gaussian forecast.
    return sigma * (z * (2 * norm.cdf(z) - 1) + 2 * norm.pdf(z) - 1 / np.sqrt(np.pi)) #Closed form Gaussian CRPS.
```

```
ensemble_members = df.drop(columns=["obs"]) #Drop the observation column for training.
observations = df["obs"]
```

```
split = int(len(df) * 0.8) #Splitting the data 80% train 20% test.
```

```
X_train = ensemble_members.iloc[:split]
y_train = observations.iloc[:split]
```

```
X_test = ensemble_members.iloc[split:]
y_test = observations.iloc[split:]
```

```
def emos(params, observations, ensemble_members): #Function for optimizing a,b,c and d.
    a, b, c, d = params
```

```
    ens_mean = ensemble_members.mean(axis=1).to_numpy() #Mean of ensemble members.
    spread2 = ensemble_members.var(axis=1).to_numpy() #Spread of ensemble members.
    obs = observations.to_numpy() #Keeping every thin in array so nothing breaks with pandas.
```

```
    mu = a + b * ens_mean #Estimation of the predictive mean.
    sigma2 = c + d * spread2 #Estimation of the predictive variance.
```

```
    neg_log_loss = 0.5 * np.log(2 * np.pi * sigma2) + ((obs - mu) ** 2) / (2 * sigma2) #Negative log loss.
    return np.mean(neg_log_loss) #We are optimizing the parameters a,b,c and d by minimizing negative log loss.
```

```
params = np.array([0.0, 1.0, 1.0, 1.0]) #Start values for the parameters.

res = minimize(
    emos,
    x0=params,
    args=(y_train, X_train),
    method="L-BFGS-B",
) #scipy.minimize for minimizing the loss function.

a, b, c, d = res.x #The resulting parameters, which minimized the loss function the most.

#Testing how the EMOS perform on a held out test subset.
ens_mean_test = X_test.mean(axis=1)
spread2_test = X_test.var(axis=1)

mu_test = a + b * ens_mean_test
sigma2_test = c + d * spread2_test
sigma_test = np.sqrt(sigma2_test)

#Calculating PIT and plotting the PIT diagram
pit = pd.Series(norm.cdf(y_test, loc=mu_test, scale=sigma_test))

pit.plot(kind="hist", bins=20)
plt.xlabel("PIT")
plt.ylabel("Frequency")
plt.title("EMOS PIT Histogram")
plt.show()

crps_raw_each = crps_ensemble(y_test.to_numpy(), X_test.to_numpy()) #CRPS for each row in the raw ensemble forecast.
crps_raw = np.mean(crps_raw_each) #Average CRPS across all test rows for the raw ensemble.

crps_emos_each = crps_gaussian(y_test.to_numpy(), mu_test.to_numpy(), sigma_test.to_numpy()) #CRPS for each row in the EMOS
crps_emos = np.mean(crps_emos_each) #Average CRPS across all test rows for the EMOS forecast.

print("Raw ensemble mean CRPS:", crps_raw) #Print the mean CRPS for the raw ensemble.
print("EMOS mean CRPS:", crps_emos) #Print the mean CRPS for the EMOS forecast.
print("CRPS improvement (raw - EMOS):", crps_raw - crps_emos) #Positive value means EMOS improved compared to the raw ensemble
```

#CRPS clearly reduced compared to the raw CRPS, PIT diagram somewhat flat but there is a long way for a perfectly calibrated

